

# PROTOCOLE MOBILE

*Documentation technique*

---

## **Documentation technique — version 1.2**

Composant mobile du projet Salufolio

**Pierre-Henri Giraud**

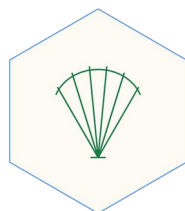
Ingénieur informaticien retraité · Les Useres (Castellón), Espagne

Juillet 2026

*Logiciel libre — GNU General Public License v3*

*salufolio.com · github.com/caracole/Salufolio · salufolio@proton.me*

*Décrit la version 2026.8.1.24-SAT. Pour l'usage quotidien, voir le Mode d'emploi.*



*À qui s'adresse ce document ?*

| Document                       | S'adresse à                                               | Niveau requis                                   |
|--------------------------------|-----------------------------------------------------------|-------------------------------------------------|
| Guide d'installation           | Qui installe l'application                                | Aucun                                           |
| Manuel d'utilisation           | Le soignant, au quotidien                                 | Aucun                                           |
| <b>Documentation technique</b> | <b>Qui maintient ou fait évoluer le code</b>              | <b>Programmation (JS, HTML, client-serveur)</b> |
| Document académique            | Chercheurs et professionnels de santé curieux du pourquoi | Aucun — pas de prérequis technique              |

## 1. Architecture

Le Protocole est un fichier HTML unique, de quelques dizaines de kilo-octets, contenant sa structure, sa présentation, sa logique et ses quatre langues. Aucune dépendance externe, aucun cadre de programmation, aucune compilation. Le fichier est à la fois le programme, le paquet de distribution et la documentation.

**Principe d'entretien hérité de la doctrine de 1966 : le comportement ne s'écrit pas dans le code, il se déclare dans des tables que le code parcourt. Pour modifier le comportement, on modifie une table — pas une fonction.**

## 2. Écran, zoom et débordement

Le corps de la page s'affiche avec un facteur d'agrandissement conservé dans `mfp_zoom`, dont la valeur par défaut est 1,25 ; les boutons A- et A+ le déplacent entre 0,8 et 1,4. La conséquence mérite d'être écrite, car elle a coûté cher : la largeur disponible est la largeur de l'écran divisée par ce facteur.

**Une ligne réclamant 540 points se trouvait invisible sur n'importe quel téléphone au facteur par défaut, alors qu'elle s'affichait parfaitement sur une tablette. Toute ligne de commandes porte désormais flex-wrap. La hauteur souffre du même mal : à l'intérieur du corps agrandi, 100vh ne vaut pas l'écran mais l'écran multiplié par le facteur, si bien qu'une fenêtre bornée en vh tronquait son propre bouton. Les fenêtres à liste sont bornées en pixels réels divisés par le facteur.**

Règle : aucune ligne ni fenêtre nouvelle sans l'éprouver à 360 points au facteur 1,4.

## 2.bis La hauteur utile se demande, elle ne se suppose pas

Une carte bornée en 88vh débordait sur un petit téléphone : dans un corps agrandi par le zoom, 88vh valent plus que l'écran. Et sur un mobile les barres du navigateur se rétractent au défilement, de sorte qu'aucune constante ne peut les suivre. Le navigateur expose la mesure exacte :

```
function pantallaUtil() {
  const z = parseFloat(localStorage.getItem('mfp_zoom') || 1);
  const vv = window.visualViewport;
  return { an: Math.round((vv ? vv.width : innerWidth) / z),
          al: Math.round((vv ? vv.height : innerHeight) / z) };
}
```

Tout est rendu en points de CONTENU, c'est-à-dire divisé par le facteur d'agrandissement : à l'intérieur du corps agrandi, un point d'écran ne vaut pas un point de contenu. La carte se remesure quand l'écran tourne et quand les barres se rétractent.

## 3. Stockage et casiers

Tout est conservé dans le localStorage du navigateur, sous le préfixe mfp\_. Chaque patient a son propre casier : ses clés portent son identifiant. Une table déclare ce qui appartient au patient ; le reste appartient à l'aidant et demeure commun.

```
const CLAVES_PACIENTE = [
  'mfp_patient', 'mfp_sip', 'mfp_meds', 'mfp_tomas', 'mfp_puntuales',
  'mfp_obs', 'mfp_constantes', 'mfp_material', 'mfp_material_cfg',
  'mfp_asist', 'mfp_avisos', 'mfp_horas', 'mfp_inicio' ];

function _k(k) {
  const a = getActivo();
  return (a && CLAVES_PACIENTE.indexOf(k) >= 0)
    ? ('mfp_' + a + '_' + k.slice(4)) : k;
}
```

**lsGet et lsSet sont les deux seules portes du stockage, et c'est là — en un seul endroit — que le préfixe se pose. Les cent endroits du programme qui écrivent une prise ou une observation n'ont rien eu à apprendre.**

| Clé                 | Contenu                                                                        |
|---------------------|--------------------------------------------------------------------------------|
| mfp_pacientes       | La liste des patients : identifiant, nom, SIP. Elle fait foi pour le nom       |
| mfp_activo          | Le patient courant. En sessionStorage : il appartient à l'ONGLET               |
| mfp_<id>_tomas      | Prises régulières du patient, groupées par jour                                |
| mfp_<id>_obs        | Ses observations horodatées : registres du corps, chips, occupations, urgences |
| mfp_<id>_constantes | Ses mesures horodatées                                                         |

| Clé                                   | Contenu                                                     |
|---------------------------------------|-------------------------------------------------------------|
| mfp_<id>_material                     | Son usage du matériel : sessions (_uso) et débits (_litros) |
| mfp_lang, mfp_theme, mfp_zoom, mfp_pc | De l'aidant — communs à tous les patients                   |
| mfp_feedback                          | Commentaires de l'aidant. Ne quitte jamais l'appareil       |

**Avertissement — le stockage l’emporte sur le code. Une valeur par défaut corrigée dans le programme ne s’applique pas à un appareil qui a déjà mémorisé l’ancienne. Toute évolution d’une valeur déclarée exige une migration explicite et douce à la lecture.**

### 3.1 Migration d’un appareil ancien

Un dossier antérieur à la liste devient le patient n° 1 au premier démarrage : ses clés passent dans son casier et les anciennes sont retirées. Ne s’exécute qu’une fois et ne perd rien.

### 3.2 Pourquoi plusieurs fichiers ne suffisent pas

**Copier le programme sous trois noms ne sépare rien : le navigateur range par ORIGINE, non par nom de fichier. Trois copies servies depuis la même adresse écrivent dans le même casier. Seuls des ports différents donneraient des origines différentes — au prix de trois serveurs et de trois adresses.**

## 4. Le changement de patient

Le mécanisme a d’abord été éprouvé sur une maquette autonome, traces à l’appui, avant de toucher au programme. Il compte trois temps et aucun rechargement de page.

```
function cambiarPaciente(id) {
  guardarActual();      // ① poser ce qui est à l’écran
  setActivo(id);        // ② changer de casier
  restaurarCampos();    // ③ relire le nouveau et repeindre
  render(); renderDatos(); renderPacientes();
}
```

**Le troisième temps est celui qui manquait : le programme n’avait jamais eu d’étape de RESTAURATION, il s’en remettait au rechargement de la page. Or un navigateur, en rechargeant, restaure les champs d’un formulaire — de sorte que les cases du patient précédent reparaissent sur le dossier du suivant. Les données étaient bonnes ; l’écran mentait.**

restaurarCampos rouvre la fiche du jour au lieu d’en créer une neuve, et retrouve la note libre grâce à une marque (libre:true) posée par upsertObs.

## 4.1 Le patient appartient à l'onglet

mfp\_activo est conservé dans sessionStorage, cloisonné par onglet et survivant au rechargement ; localStorage ne garde que le dernier choisi, pour qu'un onglet neuf s'ouvre là où l'on s'est arrêté. Deux onglets peuvent ainsi porter deux patients sans se marcher dessus — ce qui produisait auparavant des observations attribuées au mauvais malade.

## 5. L'écriture à la frappe

**Doctrine : un changement d'état d'un paramètre s'enregistre obligatoirement, sans confirmation. L'événement « change » ne suffit pas : il n'arrive qu'à la SORTIE du champ, si bien que rien ne s'écrit tant que le doigt y est, et que tout s'évapore si l'écran bascule avant.**

Une seule écoute déléguée sur le document, et une table qui dit ce que fait chaque champ. Ajouter un champ demain, c'est ajouter une ligne.

```
const CAMPOS_GUARDA = {
  'c-tas': () => upsertConst(), /* ... les sept constantes ... */
  'nota-libre': () => upsertObs(),
  'cfg-patient2': e => renombrarActivo(e.value, null),
  'cfg-sip': e => renombrarActivo(null, e.value),
  'cfg-pc': e => setPc(e.value)
};
document.addEventListener('input', guarda, true);
document.addEventListener('change', guarda, true);
```

Les deux sont écoutés : « input » fait le travail, « change » rattrape les listes déroulantes et les sélecteurs de date sur les navigateurs qui n'émettent pas « input ». Les fonctions appelées sont des upsert : écrire cent fois ne crée pas cent fiches. Et l'écoute n'avale pas les erreurs : un enregistrement qui échoue en silence est pire qu'un enregistrement qui manque.

## 6. Les tables déclaratives

| Table           | Gouverne                                                                                       |
|-----------------|------------------------------------------------------------------------------------------------|
| MOMENTOS        | La taxinomie unique : petit-déjeuner, déjeuner, dîner, nuit, à la demande                      |
| CHIP_TIPOS      | Le vocabulaire d'observation, dans l'ordre de l'écran                                          |
| CHIP_OPC        | Pour chaque événement : la question, les options et la FORME COMPLÈTE de chaque état résultant |
| CHIP_IDIOMAS    | La traduction des événements et de leurs formes complètes                                      |
| BEBIDAS         | Les boissons de l'hydratation                                                                  |
| U112            | Les quatre temps de l'épisode d'urgence, dans leur ordre                                       |
| MATERIAL_DEF    | Les appareils et leur type de relevé                                                           |
| CAMPOS_GUARDA   | Quelle fonction enregistre chaque champ de saisie                                              |
| CLAVES_PACIENTE | Quelles clés appartiennent au patient et lesquelles à                                          |

| Table   | Gouverne                                                   |
|---------|------------------------------------------------------------|
|         | l'aidant                                                   |
| IDIOMAS | Toutes les chaînes de l'interface, dans les quatre langues |

## 6.1 Les règles d'or

- L'identifiant est toujours stocké en espagnol ; seul l'affichage est traduit. Sans cette règle, un fichier produit en français et lu en valencien serait illisible pour le dossier.
- Les formes sont DÉCLARÉES, jamais composées. Composer « base + suffixe » produisait « Dolor pierna izquierdo » — masculin sur un mot féminin.
- Seule exception, et seulement parce que les deux formes sont écrites : le nombre choisit entre singulier et pluriel (clé et clé\_1). Il choisit, il ne fabrique pas.
- Aucune chaîne en dur hors de la table IDIOMAS.

## 6.2 Ajouter un événement

Trois pas, dont aucun dans le code de l'interface : ajouter l'identifiant espagnol à CHIP\_TIPOS, déclarer ses options et ses formes complètes dans CHIP\_OPC, et ses traductions dans CHIP\_IDIOMAS. Une famille à choix multiple se marque par multi:true, et ses options exclusives par excl.

### 6.bis La fréquence des médicaments

Le moment du jour ne suffit pas : une ampoule mensuelle déclarée au petit-déjeuner paraîtrait chaque matin. Une seconde table déclare les fréquences, et une fonction dit si le médicament est dû aujourd'hui.

```
const FRECUENCIAS = {
  diaria: { k:'fr_diaria', pide:null },
  alterna: { k:'fr_alterna', pide:null }, // ancla = date de déclaration
  semanal: { k:'fr_semanal', pide:'sem' },
  mensual: { k:'fr_mensual', pide:'mes' } };
```

```
function tocaHoy(m, fecha) { ... } // l'écran du jour ne demande que cela
```

Un médicament du 31 ne disparaît pas en février : on prend le minimum entre le jour déclaré et le dernier du mois. Rien ne se perd. Et le champ du jour ne se montre que pour les fréquences qui l'exigent — ne pas tenter le diable.

### 6.ter Le rythme irrégulier

Une clé de plus dans la mesure, et rien d'autre : arritmia: true. Le format 2 accepte les clés nouvelles, et une clé absente signifie « non observé », ce qui est exactement juste. Le bouton porte le symbole d'éclair des tensiomètres : l'écran de l'appareil et celui du téléphone disent la même chose.

## 7. La nature du moment et la relecture

Toute observation n'a pas le même rapport au temps. Une chute survient à une heure précise : l'heure EST le fait. « Occupation : rien » ne survient à aucune heure. Une table déclare cette nature, et une seule :

```
const CUANDO_JORNADA = [
  'Somnolencia', 'Alimentación', 'Ocupación', 'Agitación / inquietud',
  'Buen día ✨', 'Dolor cabeza', ... , 'Hinchazón piernas' ];

function cuandoDe(etiq) {           // 'instante' | 'jornada'
  // les formes complètes héritent de la nature de leur base ;
  // ce qui n'est pas déclaré est un instant (registres du corps, 112, notes)
}
```

Un seul indicateur a été choisi délibérément, non deux. « On garde l'heure, mais elle n'est pas révisable » ne signifie rien : si l'heure ne signifie rien, elle ne doit pas être écrite. La doctrine de Lavoisier s'applique aussi en sens inverse — ne pas enregistrer ce qu'on n'a pas observé.

### 7.1 Ce qui reste à relire

```
function pendientesRevision() {
  return getObs().filter(o => o.hora && !o.rev
    && cuandoDe(o.tipos[0]) === 'instante');
}
```

Le tampon est une marque de date dans l'observation elle-même : `o.rev = "2026-07-31T09:00"`. Rien de plus. Une observation sans tampon n'est pas suspecte : elle n'est simplement pas encore passée entre les mains de quelqu'un.

La vitrine de ces attentes a changé le 3 août : le volet « Últimos 7 días » du panneau Datos — sept était arbitraire — a cédé la place à « Días por transmitir », qui groupe `pendientesRevision()` par date. Chaque jour est un bouton : il ouvre la relecture filtrée sur ce seul jour (`pantallaRevision(true, dia)`), et à sa fermeture le jour disparaît de la liste. Ce qui reste affiché est, exactement, ce qui reste à faire ; la lecture de l'histoire appartient au Resumen de la barre du bas.

### 7.2 La séquence de fermeture

Tamponner, enregistrer, et envoyer si le pont répond. La copie locale n'est pas conditionnelle ; seul l'envoi l'est. Un aidant qui a relu sa journée ne doit pas perdre ce travail parce qu'un ordinateur est éteint.

Depuis la v2026.8.1.23-SAT, la séquence n'emploie plus aucune minuterie. Sur Android, `a.click()` peut ouvrir le sélecteur du système (« où enregistrer ? ») ; tant qu'il est ouvert, la page perd le premier plan et un avis parlerait sous lui. La parole et l'envoi partent au retour au premier plan : `alVolverAlPrimerPlano(cb)` appelle immédiatement si `document.hasFocus()` ; sinon, il attend le premier focus ou `visibilitychange` — les deux issues du sélecteur, enregistré ou annulé, sont couvertes. Le dernier avis est toujours le dernier mot de la séquence, et le volet des jours se repeint à la fin.

## 8. L’emblème et les fenêtres

### 8.1 Une seule main dessine

Le même emblème existait trois fois dans le fichier, avec trois géométries écrites à la main : celle de l’en-tête, celle du blason d’accueil et celle de la fonction de dessin. Trois vérités qui auraient divergé au premier retouchage. Désormais les rayons et l’arc se déclarent une fois :

```
const CONCHA_RAYOS = [[4,124],[24,94],[51,70],[84,55],[120,50], ... ];
const CONCHA_ARCO = 'M4,124 Q2,101 24,94 ...';

conchaSVG(couleur, taille) // la coquille seule ; en petit, un rayon sur deux
blasonSVG(taille, couleur) // l'hexagone et LA MÊME coquille, mise à l'échelle
```

La sonde ne repeint pas trait par trait : elle redessine. Trois états — vert si le pont répond, or en pénombre s’il ne répond pas, or plein s’il n’y a pas de pont à chercher. La maison ne change pas de couleur parce qu’on ne lui répond pas : elle baisse la lampe.

*Note héraldique du 3 août : l’emoji 🐚 est une coquille spiralée — un gastéropode —, non la coquille du pèlerin. Là où l’emblème doit se montrer hors du programme (le panal), on dessine un pecten en SVG : l’éventail des stries convergeant vers la charnière, fidèle au blason.*

### 8.2 Ne jamais enchaîner deux fenêtres par une minuterie

Un écran se rouvrirait quatre cent vingt millisecondes après l’ouverture d’un correcteur, c’est-à-dire par-dessus lui. Le remède n’est pas d’allonger le délai mais de le supprimer : la première fenêtre prévient quand elle se referme.

```
function corregirHoraEv(idx, despues) {
  const cerrar = () => { ov.remove(); if (despues) despues(); };
  // les DEUX issues -valider et annuler- appellent cerrar()
}
```

Et l’on tient la liste des étages : 94 à 99. Aucune fenêtre ne doit pouvoir enterrer une autre.

## 9. Le format d’échange

Le fichier exporté est un JSON conçu pour être lisible par un humain. Les événements sont groupés par jour et la date sert de clé ; aucune clé vide n’est écrite ; le matériel est décrit par sessions observées et non par durées calculées.

```
{
  "version": "Salufolio-Protocolo 2026.8.1.24-SAT",
  "formato": 2,
  "paciente": { "nombre": "...", "sip": "..." },
  "tomas": { "2026-07-29": [
    { "momento": "desayuno", "hora": "08:15" },
    { "med": "Paracetamol 1 g", "hora": "16:40" } ] },
  "material": { "uso": { "oxigeno": { "2026-07-29": [
    { "inicio": "14:05", "fin": "16:40",
```



```
"litros": [ {"hora":"14:05","lpm":2} ] } ] } }
```

Un envoi, un patient : le dossier de destination est par patient, et l'on doit pouvoir remettre celui de l'un sans celui de l'autre. La lecture accepte le format 2 et tolère le format 1 des versions anciennes ; les étiquettes composées d'autrefois restent reconnues.

*Prévu et non encore écrit : un format 3, simple enveloppe pour la sauvegarde de toute la maison, où chaque élément du tableau est un corps de format 2 tel quel.*

## 10. Le pont et la boîte de réception

| Élément            | Valeur                                         |
|--------------------|------------------------------------------------|
| Port               | 7777                                           |
| Application servie | /protocolo                                     |
| Envoi              | POST /api/protocolo                            |
| Liste des reçus    | GET /api/protocolo/lista                       |
| Dossier d'entrée   | protocolo-entrada                              |
| Nom des fichiers   | MFProtocolo_<identifiant>_v<date>_<heure>.json |

L'envoi et le sondage se font en mode no-cors, sans lire de réponse, avec un en-tête de texte simple pour éviter la requête préalable. Ce qu'on y gagne, c'est que l'envoi marche de partout, même depuis une copie locale ; ce qu'on y perd, c'est l'accusé de réception.

**Le serveur vérifie la signature du paquet avant de l'accepter. Cette vérification a jadis causé une panne complète : un changement de nom du projet a invalidé une signature écrite en dur. D'où la doctrine : toute constante susceptible de changer appartient à un fichier de configuration, jamais au code.**

Le PC ne renomme plus personne : la liste du téléphone fait foi. Son étiquette n'est adoptée que si la maison est encore vide.

### 10.1 Les trois horloges

L'heure de mesure vit à l'intérieur de la donnée et c'est la seule qui soit clinique ; l'heure d'envoi figure dans le nom du fichier ; l'heure d'insertion est un tampon administratif. Les trois sont conservées, aucune ne remplace l'autre.

### 10.bis La chaîne vide est une réponse valable

L'adresse du pont connaît trois cas, et non deux. Servi par le pont local, elle vaut la chaîne vide — « même origine » —, ce qui est exact. Depuis une copie locale ou depuis le dépôt public, elle vaut l'adresse configurée. Et lorsque rien n'est connu, elle vaut null.

```
function baseSalufolio() {
  const esLocal = /^(localhost$|127\.|10\.|192\.168\.|172\.(1[6-9]|2[0-9]|
3[01])\.)/
    .test(location.hostname);
  if (location.protocol.indexOf('http') === 0 && esLocal) return '';
  const pc = String(getPc() || '').trim().replace(/\/+$/, '');
  return pc ? pc : null; // null, jamais ''
}
```

Trois fonctions la testaient par une épreuve de véracité, et une chaîne vide est fausse. Conséquence : servi par l'ordinateur — le cas souhaitable — la sonde ne sondait jamais, l'emblème ne s'allumait pas et l'envoi répondait « dirección? ». Le pont ne fonctionnait que depuis une copie locale. La correction consiste à tester `base === null`, et cela seul.

Second défaut de même origine : `indexOf('http') === 0` est vrai pour https aussi. Depuis le dépôt public, le programme croyait devoir s'envoyer à lui-même. D'où le contrôle d'origine locale.

## 10.ter Ce qui ne peut pas marcher ne se montre pas

Le bouton d'envoi ne paraît que si `_pcVivo === true` ; à sa place se lit la raison. Le bouton de relecture annonce le nombre de lignes à relire, ou dit « Enregistrer et envoyer » s'il n'y en a aucune, et n'ouvre alors aucun écran. Et le champ de l'adresse de l'ordinateur se cache lorsque le programme est servi par le pont : on ne demande pas ce qu'on sait déjà.

## 10.quater La chaîne d'installation : la route, le code QR et le scanner

Installer le Protocolo sur un appareil neuf était le dernier geste qui exigeait de taper une adresse. Depuis le 3 août, la chaîne entière vit dans la maison : la route préconfigure, le code QR épargne le clavier, et le programme lui-même sait photographier.

La route `/protocolo/instalar` (serveur v1.7.0) lit `app.html`, remplace `const MFP_PC_DEF = "` par l'adresse réelle du pont — obtenue par socket, valable sous Linux, Windows ou Mac — et sert le résultat en `application/octet-stream` avec `Content-Disposition: attachment` et le nom de baptême `Salufolio_Protocolo_movil.html`. Deux conséquences : le navigateur ne bloque plus le téléchargement (« impossible de télécharger de façon sécurisée » était le prix du `text/html` servi en HTTP clair), et la copie locale naît configurée : elle connaît son pont sans que personne n'écrive rien. Le bouton « ↓ Oui, installer » choisit le chemin selon l'origine : servie par le pont, l'application demande le fichier à la route ; depuis `file://` ou le dépôt public, le programme se lit lui-même.

Le code QR montre l'adresse du jour. Il vit en deux endroits : l'outil d'atelier `qr.sh` (Linux : il lit l'IP du moment et engendre la page) et — vivant — dans l'alvéole Protocolo du panal : `sondaPuente()` interroge `/api/ip` au chargement (garde de 1,6 s), la coquille-pecten se peint en vert si le pont répond et en or sinon, et le code s'engendre à la volée avec le module `qr-js` (`qrcode-generator`, licence MIT, embarqué). Il n'est jamais périmé : l'adresse se demande, elle ne se mémorise pas. Sur la vitrine publique (https), la sonde ne peut pas joindre un serveur local : la coquille reste or et le code ne s'offre pas — ce qui ne peut pas marcher ne se montre pas. Le

lien « Abrir Protocolo » dit vrai dans les deux mondes : coquille verte, il mène à l'application servie par le pont ; sans pont, au fichier du dépôt.



Figure — La carte QR : le code du jour, l'adresse en clair, et le rappel du dernier geste (« Ajouter à l'écran d'accueil »).

ORDINATEUR (Salufolio + serveur :7777)      TÉLÉPHONE / TABLETTE  
/api/ip → panal : coquille verte + QR du jour ⇒ caméra / Lens  
/protocolo/instalar -(octet-stream, MFP\_PC\_DEF)→ copie locale préconfigurée  
→ « Ajouter à l'écran d'accueil »

Le dernier maillon regarde en sens inverse : la copie locale sait photographier le code. Le bouton , à côté du champ d'adresse, n'existe que là où il peut fonctionner : BarcodeDetector présent et contexte sûr (file://, https, localhost) — l'application servie par http://IP ne peut pas ouvrir la caméra, et c'est pourquoi elle ne le montre pas. Du code lu, on conserve la base (protocol + '/' + host), mémorisée comme une saisie manuelle, et le pont est sondé. Zéro octet embarqué : le détecteur est celui du navigateur ; l'écriture à la main reste sur la table, comme le menu papier.

## 11. Diagnostic

- Une capture d'erreur trop large ment. Un bloc générique traduisait toute défaillance par « serveur éteint » alors que le serveur fonctionnait : c'était le format qui avait évolué.
- Ce qu'on voit et ce qui est écrit peuvent diverger en silence. Tout état affiché doit se relire dans les données, jamais se garder en mémoire vive seule.
- Se méfier des comparaisons de chaînes sur les noms d'écran : une faute de frappe les rend toujours vraies et ne dit rien. La fiche des constantes se fermait en ENTRANT dessus, pour avoir comparé à « constantes » au lieu de « const ».

- Un navigateur sans tête ne restaure pas les formulaires et ne reproduit pas toutes les origines : il ne prouve rien sur ces deux points. Devant un mécanisme qui résiste, faire une maquette autonome avec ses traces avant de toucher au programme.

## 12. Entretien

| Opération                          | Où elle se fait                                           |
|------------------------------------|-----------------------------------------------------------|
| Ajouter un événement               | CHIP_TIPOS, CHIP_OPC et CHIP_IDIOMAS                      |
| Ajouter une boisson                | BEBIDAS, plus sa clé dans IDIOMAS                         |
| Ajouter un champ de saisie         | CAMPOS_GUARDA, plus sa restauration dans restaurarCampos  |
| Ajouter une clé du patient         | CLAVES_PACIENTE — sinon elle sera commune à tous          |
| Ajouter ou changer une chaîne      | IDIOMAS, dans les quatre langues à la fois                |
| Changer port, dossier ou signature | Fichier de configuration du serveur — jamais dans le code |
| Publier une version                | Mettre à jour MFP_VERSION et recopier le fichier servi    |

*En cas de divergence, le fichier source fait foi : étant unique et lisible, il contient sa propre vérité.*